

AML Integrated Reporting

A Demonstration of LLM-Assisted Pathology Reporting

How to run the demo, what it shows, and how it could work in real clinical practice without sending patient data to the cloud.

What this is

This package contains a complete teaching case. One fictional patient has had a bone marrow workup, and four separate laboratory reports have been produced: morphology, flow cytometry, cytogenetics, and molecular sequencing. The demo shows how a large language model can take those four separate reports and assemble them into a single integrated diagnostic report, with every statement traceable back to its source.

The patient and all data are invented for teaching. Nothing here is real patient information.

What is in the package

File	What it is
01_bone_marrow_morphology.pdf	Source report 1 of 4
02_flow_cytometry.pdf	Source report 2 of 4
03_cytogenetics_fish.pdf	Source report 3 of 4
04_molecular_ngs.pdf	Source report 4 of 4 (from a separate reference lab)
prompt_extractor.txt	Stage 1 instructions for the model
schema_extractor_output.json	The structured format Stage 1 must produce
prompt_integrator.txt	Stage 2 instructions for the model
05_GOLD_STANDARD_integrated_report.pdf	The pathologist-written answer key, with instructor notes
00_DEMO_GUIDE.pdf	This document

The idea in one paragraph

In modern hematopathology the diagnosis often depends on combining several separate reports. No single report is sufficient on its own. The morphology report may be unsure about lineage. The cytogenetics report may look normal and almost reassuring. The finding that actually defines the disease may be sitting alone in the molecular report. Reading these one at a time, a clinician can miss how they fit together. An integrated report solves that by synthesizing all of them into one coherent interpretation. This demo shows a model doing the first-pass assembly, with a built-in evidence trail so a pathologist can check the work quickly.

PART 1 • Running the Demo

Seven steps. Plan for about fifteen minutes if you run it live. Before you start, have these open and ready: the four source PDFs, the extractor prompt, the schema file, the integrator prompt, and the gold standard (keep that last one hidden until Step 6).

Step 1 — Show the problem

Open the four source PDFs one at a time. Make the point that this is one patient and one bone marrow specimen, but four reports, each issued by a different lab on a different day. They look nothing alike because they come from different laboratory systems.

Say: "No single one of these is a complete diagnosis. The morphology report isn't sure about lineage. The cytogenetics report is completely normal and almost looks reassuring. The finding that actually defines the disease is sitting alone in the molecular report. A clinician reading these one at a time can miss how they fit. That is the problem an integrated report solves."

Step 2 — Explain the two-stage approach

Tell them you are not going to ask the model to "read four PDFs and write a report" in one shot, because that is where models blur things together and invent detail. Instead it is two stages. Stage one extracts the facts into a structured form. Stage two reasons over that structure to write the report. Separating the two means every claim in the final report can be traced back to a source.

Step 3 — Run the extractor

Open a new chat with the model. Attach all four source PDFs. Paste in the contents of prompt_extractor.txt. Send it. What comes back is a single block of structured data (JSON): the findings from each report, plus a section listing where the reports agree, where they appear to conflict, and which findings exist in only one report. Show that output briefly.

Say: "Notice that nothing has been interpreted yet. It has just been organized, and every finding is tagged with which PDF it came from."

Step 4 — Run the integrator

Start the next step. Paste in the contents of prompt_integrator.txt, then paste the structured data the extractor just produced. Send it. This time the model produces the actual integrated report: patient information, a summary of each of the four studies, then the key section (the integrated interpretation), then the final diagnosis. Below that it produces an evidence trace, a table mapping every sentence of the interpretation to the source that supports it.

Step 5 — Walk through what the model figured out

This is the payoff. Point to three things in the output:

- The two reports gave different blast counts, 18 percent and 22 percent. The model reconciled them instead of silently picking one.
- Morphology was not sure about monocytic differentiation. Flow cytometry settled it. The model credited flow with resolving that.
- The mutation that defines the diagnosis showed up in only one of the four reports. The model flagged that explicitly. That is the whole argument for integrated reporting in a single example.

Step 6 — Check it against the gold standard

Now reveal the gold standard PDF, the version a pathologist wrote. Put it side by side with the model's output. Compare the final diagnosis and spot-check a few evidence-trace rows. Mention the deliberate trap: there is a third mutation, DNMT3A, that is real but should not change the diagnosis. A good output mentions it but keeps it out of the diagnosis line. Show whether the model got that right.

Step 7 — Close

Summarize what they just saw: four messy reports went in, one coherent report came out, and every sentence is traceable to a source. If you want to make it land, rerun the integrator once or twice on the same structured data to show whether the output stays consistent.

Two practical notes

- Do a full dry run yourself first. The live extractor output will not be word-for-word identical to what you rehearsed, so know the shape of it.
- If the room is short on time, pre-run Steps 3 and 4, save the outputs, and walk through them. The live run is more convincing if you have ten minutes to spare.

The one-sentence version

Attach the four PDFs with the extractor prompt to get structured data, feed that structured data with the integrator prompt to get the report, then compare the result against the gold standard.

PART 2 • What the Case Is Designed to Test

The case was built with three deliberate features and one deliberate trap. Knowing these in advance lets you judge the model's output fairly and explain it clearly.

Discordance 1: the blast count

Morphology reports 18 percent blasts. Flow cytometry reports 22 percent. A model that quietly picks one number, or that argues the case hinges on crossing the 20 percent line, has missed the point. The correct handling is to acknowledge both numbers, explain why a manual count and a machine-gated count are expected to differ, and then note that the threshold is not the deciding factor here because a disease-defining mutation is present.

Discordance 2: the lineage

Morphology hedges on whether the leukemia shows monocytic differentiation. Flow cytometry proves that it does. The model should credit flow with resolving the question, not treat the two reports as contradicting each other.

The single-source finding

The mutations that define the diagnosis and its prognosis (NPM1 and FLT3-ITD) appear only in the molecular report. The karyotype and FISH are completely normal. This is the central argument for integrated reporting, and the model should state it plainly.

The planted trap: DNMT3A

The molecular report also lists a DNMT3A mutation. It is real and prognostically relevant, but it does not by itself define or change the disease category. A model that writes it into the final diagnosis line has failed the test. It belongs in the molecular summary and the prognostic notes, not the diagnosis.

A simple way to score the output

- Completeness: are all four modalities represented in the report?
- Accuracy: does the final diagnosis match the gold standard?
- Discordance handling: are both discordances addressed out loud, not glossed over?
- Traceability: does every sentence in the interpretation and diagnosis map to a real source?
- Lane discipline: is the DNMT3A finding kept out of the diagnosis line?

PART 3 • How This Could Work in Real Clinical Practice

The demo and a real deployment are different things. The demo pastes PDFs into a cloud chat window, which is fine for fictional teaching data and not acceptable for real patient information. The gap between the two is mostly about where the model runs and who is accountable for the output. The points below cover how a real deployment keeps protected health information inside the hospital.

The core principle

The model comes to the data, not the other way around. In a real deployment, patient data never crosses the institutional boundary. The model runs inside the hospital network, behind the same firewall as the laboratory system. Everything below serves that single principle.

Option A: an on-premises open-weight model

The cleanest approach is to run an open-weight model on hospital hardware. Several capable model families can be downloaded as static files and run entirely offline on a local server with a graphics processing unit. No external calls, no telemetry, nothing leaving the building. The tradeoff is that these models are not as strong as the best cloud models, so the pipeline design matters more. The two-stage extract-then-integrate structure shown in this demo actually helps, because each stage is a narrower task that a smaller model can do reliably. For a department-scale pilot this is one well-specified server, not a data center.

Option B: a contractually walled-off vendor model

If a pure on-premises setup is not feasible, some institutions use a vendor's model inside a private, contractually protected environment: a dedicated tenancy under a signed business associate agreement, with no retention of inputs, no training on submitted data, and full audit logging. This is not the same as the public chat interface. It is a legitimate deployment model that many health systems use. It is a heavier lift on the compliance side and a lighter lift on the hardware side. The on-premises route is the cleaner story for patient data privacy; this is the pragmatic fallback.

Connect to the lab system, not to PDFs

The demo uses PDFs because that is what can be handed to someone. A real system would pull structured results directly from the laboratory information system or the electronic health record through a standard interface. The extractor would then read discrete data fields rather than parsing a rendered document. That removes a whole class of parsing errors and keeps the data flow inside systems that already have patient-data handling, access controls, and audit trails. In practice this makes the project as much an interface project as an artificial intelligence project.

The output is a draft, not a result

This is the part that matters most and is easy to skip past. The integrated report the model produces is a draft for a pathologist to review, edit, and sign. It is never released on its own. The model does the assembly and the first-pass synthesis. The diagnostician does the verification and is accountable for the signed report. The evidence trace built into this pipeline is what makes that review fast: the pathologist can see which source supports each sentence and check the few that matter, rather than re-deriving everything by hand. The right framing is a tool that drafts, not a tool that diagnoses.

Validation and governance before any clinical use

Before touching a real case, a system like this would go through the institution's pathway for a laboratory-developed test or a clinical decision support tool. That means a validation study on real retrospective cases with known answers, measuring agreement with the final signed diagnosis and the rate of unsupported statements; documented performance characteristics; sign-off from laboratory leadership and the relevant institutional committees; and a plan for ongoing monitoring. Regulatory and accreditation bodies have expectations here. This is the part that turns a clever demo into something defensible, and it is also the part that takes the most time.

The honest summary

The demo shows the concept works. A real deployment is, roughly: an open-weight model running on a hospital server, fed by a lab-system interface rather than PDFs, producing a draft that a pathologist reviews and signs, validated and governed like any other clinical tool. The technology is the easy part. The interface work, the validation study, and the governance sign-off are where the real effort goes, and they are also what keeps patient data safe and keeps the laboratory accountable for what gets reported.

Prepared as a teaching package for the AML integrated reporting demonstration. The case, the patient, and all laboratory data are fictional and were created for teaching. Not for clinical use.